

6. Development of a secure Web Application

Barnabas Steiner

Agenda

 Keycloak

 MariaDB Least Privilege DB-User

 server.js AuthN/AuthZ für Dashboard & Controller

 controller.py AuthN/AuthZ mit Keycloak

 Web-Dashboard

 Fazit

Container Externals

Client
(Produktion Netz)

Docker-Bridge
'sensor-net'



Keycloak

HTTPS:443
(AuthN, AuthZ, RESTful API)

HTTPS:
8443
AuthN

keyCloak
(AuthN/AuthZ))

HTTPS:
8443
AuthZ

nginx
(Reverse Proxy)

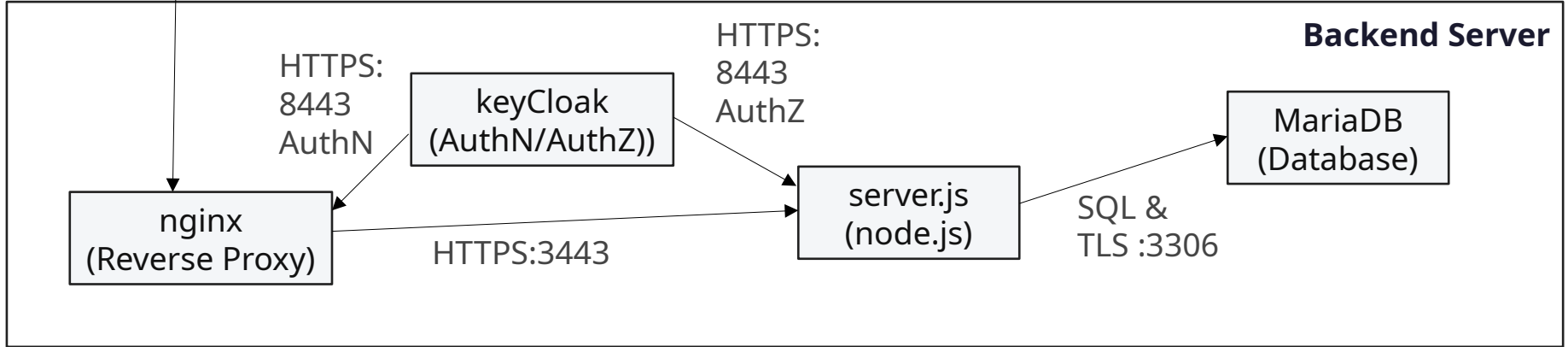
HTTPS:3443

server.js
(node.js)

SQL &
TLS :3306

MariaDB
(Database)

Backend Server



docker-compose.yml > keycloak_web Service + keycloak_data Volume

docker-compose.yml (before — nicht vorhanden)

① Neuer Service: keycloak_web (Keycloak IAM – OIDC Provider)

② Neues Named Volume: keycloak_data (persistente Keycloak-Daten)

```
106 volumes:
107   mariadb-data:
```

docker-compose.yml (after)

① Neuer Service: keycloak_web (Keycloak IAM – OIDC Provider)

```
88+ keycloak_web:
89+   image: quay.io/keycloak/keycloak:23.0.7
90+   container_name: keycloak_web
91+   environment:
92+     KC_HOSTNAME_PORT: 8443
93+     KC_HOSTNAME_STRICT: "false"
94+     KC_HOSTNAME_STRICT_HTTPS: "false"
95+     KC_PROXY: edge
96+     KC_LOG_LEVEL: info
97+     KC_METRICS_ENABLED: "true"
98+     KC_HEALTH_ENABLED: "true"
99+     KEYCLOAK_ADMIN: admin
100+    KEYCLOAK_ADMIN_PASSWORD: admin
101+    KC_HTTP_RELATIVE_PATH: /auth
102+    volumes:
103+      - keycloak_data:/opt/keycloak/data
104+      - ./keycloak/iot-realm.json:/opt/keycloak/data/import/iot-realm.json:ro
105+    command: start-dev --http-port 8080 --import-realm
106+    healthcheck:
107+      test: ["CMD-SHELL", "exec 3<>/dev/tcp/127.0.0.1/8080 && echo ..."]
108+      timeout: 10s   retries: 5   start_period: 40s
109+    ports:
110+      - "8443:8080"
111+    networks:
112+      - app-network
```

② Neues Named Volume: keycloak_data (persistente Keycloak-Daten)

```
137 volumes:
138   mariadb-data:
139+   keycloak_data:   # Keycloak persistente Daten
```

Keycloak > `iot-realm.json` > Realm-Konfiguration

// Realm Grundeinstellungen

```
1 {  
2   "realm": "iot",  
3   "enabled": true,  
4   "sslRequired": "all",  
5   "registrationAllowed": false,
```

Realm-Name

Realm aktiv

SSL für alle Verbindungen

Selbstregistrierung deaktiviert

Lese-Zugriff Dashboard

Admin-Zugriff

Sensor-Daten schreiben

// Rollen-Definition

```
6   "roles": {  
7     "realm": [  
8       { "name": "dashboard-user" },  
9       { "name": "admin-user" },  
10      { "name": "controller-ingest" }  
11    ]  
12  },
```

// Rolle → Berechtigung (Zuordnung)

`dashboard-user` → Dashboard lesen (GET /api/sensordata)

`admin-user` → Admin-Funktionen (Verwaltung)

`controller-ingest` → Sensordaten schreiben (POST /api/internal/sensordata)

Keycloak > `iot-realm.json` > Benutzer-Konfiguration

// Benutzer 1: `iotuser01` (Mensch – Dashboard-Zugriff)

```
13  "users": [ {  
14    "username": "iotuser01",  
15    "enabled": true,  
16    "emailVerified": true,  
17    "firstName": "James",  
18    "lastName": "Cookie",  
19    "credentials": [{  
20      "type": "password",  
21      "value": "password",  
22      "temporary": false  
23    }],  
24    "realmRoles": ["dashboard-user", "admin-user"]  
25  },
```

Login-Name

⚠ Nur für Entwicklung!

Benutzerrollen

Service-Account (kein Mensch)

Verknüpft mit Client

Nur Schreib-Rolle

// Benutzer 2: `service-account-controller-client` (Maschine – Controller)

```
26  {  
27    "username": "service-account-controller-client",  
28    "enabled": true,  
29    "serviceAccountClientId": "controller-client",  
30    "realmRoles": ["controller-ingest"]  
31  },  
32  ],
```

Keycloak > `iot-realm.json` > Client-Konfiguration

// Client 1: dashboard-client (Browser → OIDC Authorization Code Flow)

```
36  "clients": [ {  
37    "clientId": "dashboard-client",  
38    "enabled": true,  
39    "protocol": "openid-connect",  
40    "publicClient": true,  
41    "standardFlowEnabled": true,  
42    "directAccessGrantsEnabled": false,  
43    "serviceAccountsEnabled": false,  
44    "redirectUris": [  
45      "https://www.lab.local/*"  
46    ]  
47  },  
48 ]
```

Frontend-Client

Kein Secret (Browser-App)

Auth-Code Flow (PKCE)

Erlaubte Redirect-URL

IoT Controller (Backend)

Confidential Client (hat Secret)

Kein Auth-Code Flow

Client-Credentials Flow aktiv

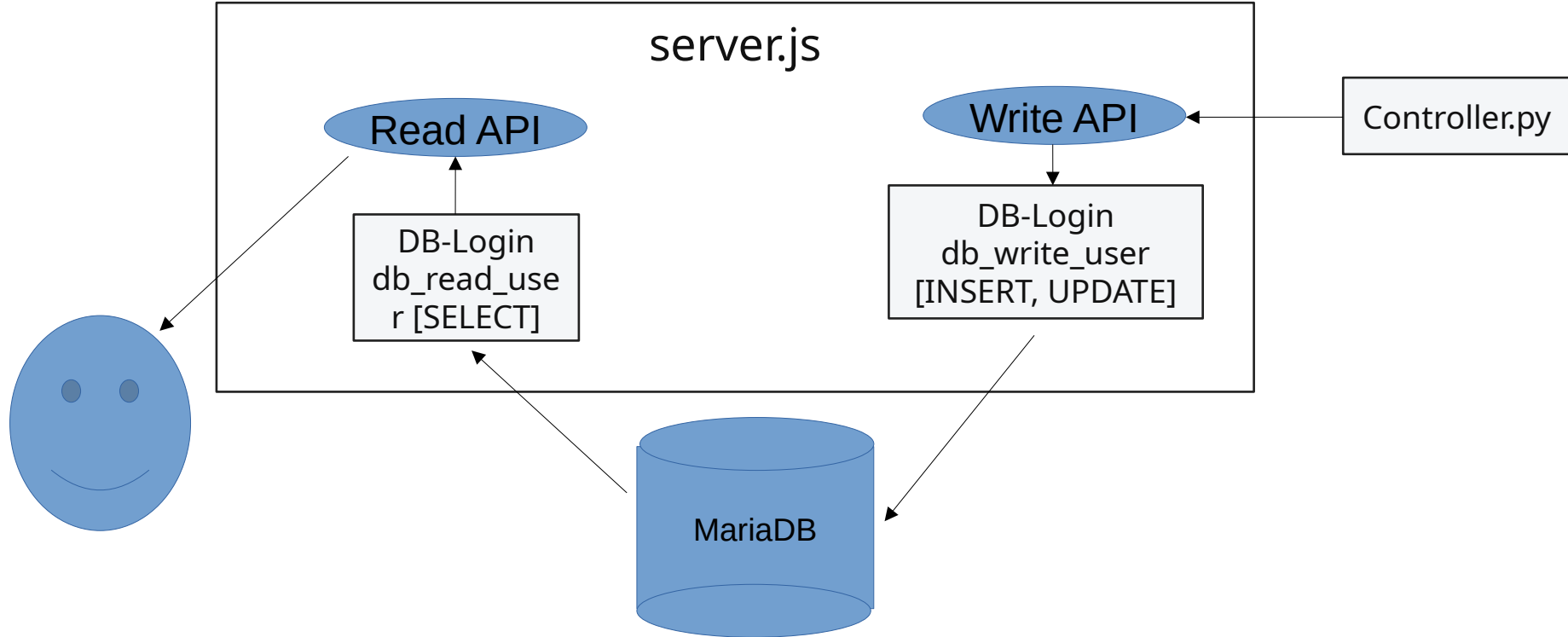
 *In Produktion ersetzen!*

// Client 2: controller-client (Maschine → Client Credentials Flow)

```
49  {  
50    "clientId": "controller-client",  
51    "enabled": true,  
52    "protocol": "openid-connect",  
53    "publicClient": false,  
54    "standardFlowEnabled": false,  
55    "directAccessGrantsEnabled": false,  
56    "serviceAccountsEnabled": true,  
57    "secret": "change-me-please"  
58  }  
59 ]
```



MariaDB Least Privilege DB-User



docker-compose.yml > Separate DB-Credentials + Keycloak Dependency

docker-compose.yml (before)

```
① webapp: environment - Split single credential → separate read/write credentials
    6     environment:
    7         - DB_HOST=mariadb
    8-        - DB_USER=websrv_write
    9-        - DB_PASSWORD=pass-0123-iot
   10        - DB_NAME=myapp
   11        - PORT=3000
   12        - API_KEY=api_key
② webapp: depends_on - Add keycloak_web health check dependency
   13     depends_on:
   14         db:
   15             condition: service_healthy
③ db: volumes - Named volume + consolidated init scripts mount
   29     volumes:
   31         - /mariadb/db:/var/lib/mysql
   32-        - /mariadb/01_config.sql:/docker-entrypoint-initdb.d/1.sql:ro
   33-        - /mariadb/02_archive_job.sql:/docker-entrypoint-initdb.d/2.sql:ro
   34-        - /mariadb/03_user.sql:/docker-entrypoint-initdb.d/3.sql:ro
   35-        - /mariadb/04_training_data.sql:/docker-entrypoint-initdb.d/4.sql:ro
   36-        -
   /mariadb/05_training_data_privileges.sql:/docker-entrypoint-
```

docker-compose.yml (after)

```
① webapp: environment - Split single credential → separate read/write credentials
    6     environment:
    7         - DB_HOST=mariadb
    8+        - DB_WRITE_USER=websrv_write
    9+        - DB_WRITE_PASSWORD=pass-0123-iot
   11+       - DB_READ_PASSWORD=pass-0123-client
   12        - DB_NAME=myapp
   13        - PORT=3000
   14        - API_KEY=api_key
   15+       - PUBLIC_DIR=app_net/public
② webapp: depends_on - Add keycloak_web health check dependency
   16     depends_on:
   17         db:
   18             condition: service_healthy
   19+        keycloak_web:
   20+            condition: service_healthy
③ db: volumes - Named volume + consolidated init scripts mount
   34     volumes:
   36         - mariadb-data:/var/lib/mysql
   37+        - ./mariadb:/docker-entrypoint-initdb.d
```

```
mariadb > 06_db_read_user > ...
```

```
06_db_read_user (neu)
```

```
1 CREATE OR REPLACE USER 'websrv_read'@'%'  
2 IDENTIFIED BY 'pass-0123-client';  
  
3 GRANT SELECT ON myapp.sensor_data  
4 TO 'websrv_read'@'%' ;  
  
5 FLUSH PRIVILEGES;
```

app-net > config > js database.js > ...

database.js (before)

```
1  require('dotenv').config(); // use .env file
2- const HOST = process.env.PORT || 3000;
3  const DB_HOST = process.env.DB_HOST
4- const DB_USER = process.env.DB_USER
5- const DB_PASSWORD = process.env.DB_PASSWORD
6  const DB_NAME = process.env.DB_NAME
7- let pool;
9- async function getPool() {
10-   if (!pool) {
11     const mariadb = await import('mariadb');
12-     pool = mariadb.createPool({
13       host: DB_HOST,
14       user: DB_USER,
15       password: DB_PASSWORD,
16       database: DB_NAME,
17       connectionLimit: 5
18     });
19   }
20-   return pool;
21 }
```

database.js (after)

```
1  require('dotenv').config(); // use .env file
2  const DB_HOST = process.env.DB_HOST
3+ const DB_WRITE_USER = process.env.DB_WRITE_USER
4+ const DB_WRITE_PASSWORD = process.env.DB_WRITE_PASSWORD
5+ const DB_READ_USER = process.env.DB_READ_USER
6+ const DB_READ_PASSWORD = process.env.DB_READ_PASSWORD
7  const DB_NAME = process.env.DB_NAME
8+ let readPool;
9+ let writePool;
11+ async function getWritePool() {
12+   if (!writePool) {
13+     const mariadb = await import('mariadb');
14+     writePool = mariadb.createPool({
15+       host: DB_HOST,
16+       user: DB_WRITE_USER,
17+       password: DB_WRITE_PASSWORD,
18+       database: DB_NAME,
19+       connectionLimit: 5
20+     });
21+   }
22+   return writePool;
23+ }
```

app-net > config > js database.js > ...

database.js (before)

```
23- module.exports = { getPool };
```

database.js (after)

```
25+ async function getReadPool() {
26+   if (!readPool) {
27+     const mariadb = await import('mariadb');
28+     readPool = mariadb.createPool({
29+       host: DB_HOST,
30+       user: DB_READ_USER,
31+       password: DB_READ_PASSWORD,
32+       database: DB_NAME,
33+       connectionLimit: 5
34+     });
35+   }
36+   return readPool;
37+ }
39+ module.exports = { getWritePool, getReadPool };
```

server.js (before)

① Import: destructured pool functions

```
1 require('dotenv').config(); // use .env file
2 const express = require('express');
4- const {getPool} = require('./config/database');
5 const PORT = process.env.PORT || 3000;
```

② POST /api/internal/sensordata - write operation → getWritePool()

```
44         if (!temperature) throw Error("Denied: ")
46-         const pool = await getPool();
47         conn = await pool.getConnection();
```

③ POST /api/internal/trainingdata - write operation → getWritePool()

```
72         if (!temperature) throw Error("Denied: ")
73-         const pool = await getPool();
74         conn = await pool.getConnection();
```

④ GET /api/internal/sensordata/latest - read operation → getReadPool()

```
95     try {
96-         const pool = await getPool();
99         conn = await pool.getConnection();
100        const sql = "SELECT * FROM sensor_data ORDER BY
timestamp..."
```

server.js (after)

① Import: destructured pool functions

```
1 require('dotenv').config(); // use .env file
2 const express = require('express');
4+ const {getReadPool, getWritePool} =
require('./config/database');
5 const PORT = process.env.PORT || 3000;
```

② POST /api/internal/sensordata - write operation → getWritePool()

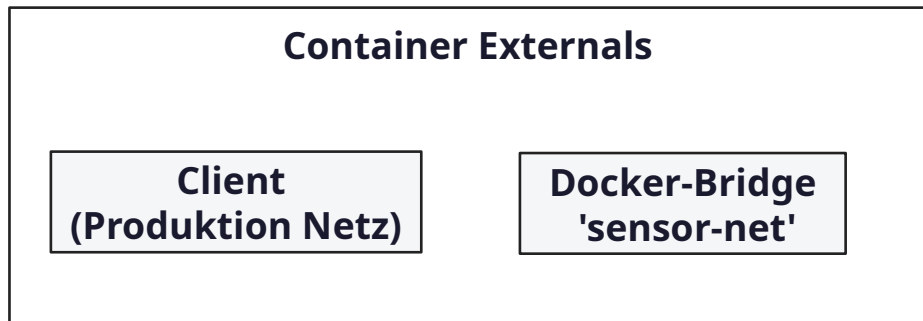
```
44         if (!temperature) throw Error("Denied: ")
45+         const pool = await getWritePool();
46         conn = await pool.getConnection();
```

③ POST /api/internal/trainingdata - write operation → getWritePool()

```
71         if (!temperature) throw Error("Denied: ")
72+         const pool = await getWritePool();
73         conn = await pool.getConnection();
```

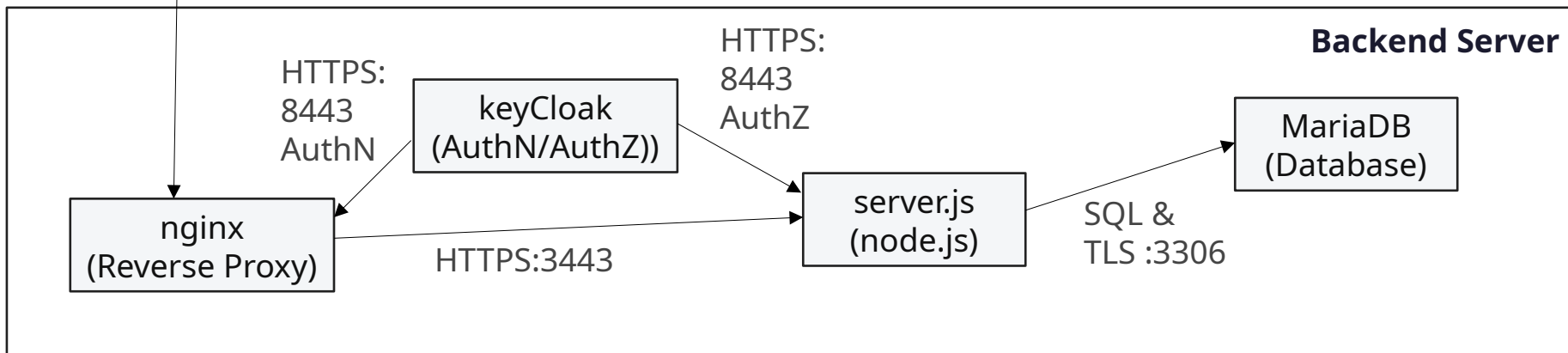
④ GET /api/internal/sensordata/latest - read operation → getReadPool()

```
94     try {
95+         const pool = await getReadPool();
98         conn = await pool.getConnection();
99         const sql = "SELECT * FROM sensor_data ORDER BY
timestamp DESC LIMIT 1"
```



server.js

AuthN/AuthZ



app-net > server.js > JWKS-Client [Teil 1/2]

server.js (before)

```
① Imports - https + jwks-rsa + jsonwebtoken + KC_JWKS_URI /
KC_BASE_URL
  4 const {getReadPool, getWritePool} =
require('./config/database');
  5 const PORT = process.env.PORT || 3000;

② JWKS-Client - Signing-Key von Keycloak via jwks-rsa abrufen
```

after

```
① Imports - https + jwks-rsa + jsonwebtoken + KC_JWKS_URI /
KC_BASE_URL
  4 const {getReadPool, getWritePool} =
require('./config/database');
  5 const PORT = process.env.PORT || 3000;
  6+ const https = require('https');
 11+ const jwksClient = require('jwks-rsa');
 12+ const jwt = require('jsonwebtoken');
 14+ const KC_JWKS_URI = process.env.KC_JWKS_URI;

② JWKS-Client - Signing-Key von Keycloak via jwks-rsa abrufen
 17+ const client = jwksClient({
 18+   jwksUri: KC_JWKS_URI,
 19+   requestAgent: new https.Agent({ ca: undefined,
rejectUnauthorized: true }),
 23+ });
 25+ const getKey = (header, callback) => {
 26+   client.getSigningKey(header.kid, (err, key) => {
 30+     const signingKey = key.getPublicKey();
callback(null, signingKey);
 33+ }); // end getKey
```

app-net > server.js > authenticateToken-Middleware (JWT AuthZ) [Teil 2/2]

server.js (before)

③ Neue Middleware authenticateToken - Bearer-Check → JWT-Verify → Audience → Rolle

after

③ Neue Middleware authenticateToken - Bearer-Check → JWT-Verify → Audience → Rolle

```
36+ const authenticateToken = (req, res, next) => {
37+   const authHeader = req.headers.authorization || "";
39+   if (!authHeader.startsWith("Bearer ")) return res.sendStatus(401);
51+   const token = authHeader.split(" ")[1];
56+   jwt.verify(token, getKey, validationOptions, (err, decodedPayload) => {
58+     if (err) return res.status(403).json({ error: "Forbidden",
62+       message: "Token verification failed (signature, expiration or issuer invalid)." });
67+     const expectedAudience = 'dashboard-client';
73+     const audienceOk = (azp === expectedAudience) || (aud === expectedAudience) || Array.isArray(aud)...;
79+     if (!audienceOk) return res.status(403).json({ error: `Audience '${expectedAudience}' missing` });
85+     const requiredRole = 'dashboard-user';
87+     const roles = decodedPayload.realm_access?.roles || [];
89+     if (requiredRole && !roles.includes(requiredRole)) return res.status(403).json({...});
96+     req.user = decodedPayload; next();
99+   }); // end jwt.verify
100+ }; // end authenticateToken
```


app-net > server.js > GET /api/sensordata mit authenticateToken für Dashboard-Chart

server.js (before)

① Neuer Endpoint GET /api/sensordata – authenticateToken (JWT) statt authenticateApiKey

② Vergleich: welcher Endpoint nutzt welche Auth-Middleware

```
POST /api/internal/sensordata
authenticateApiKey // API-Key (x-api-key) – Controller schreibt

POST /api/internal/trainingdata
authenticateApiKey // API-Key (x-api-key) – Controller schreibt

GET /api/internal/sensordata/latest
authenticateApiKey // API-Key (x-api-key) – Controller liest
```

after

① Neuer Endpoint GET /api/sensordata – authenticateToken (JWT) statt authenticateApiKey

```
206+ // For Dashboard chart
207+ app.get('/api/sensordata', authenticateToken, async
(req, res) => {
209+   try {
210+     const pool = await getReadPool();
211+     conn = await pool.getConnection();
214+     const sql = "SELECT * FROM sensor_data ORDER BY
timestamp DESC";
215+     const rows = await conn.query(sql);
217+     if (rows.length === 0) return
res.status(200).json({message: "No data available."});
220+     res.status(200).json(rows[0]);
221+   } catch (err) { res.status(500).json({error:
err.message});
223+   } finally { if (conn) conn.release(); }
226+ });
```

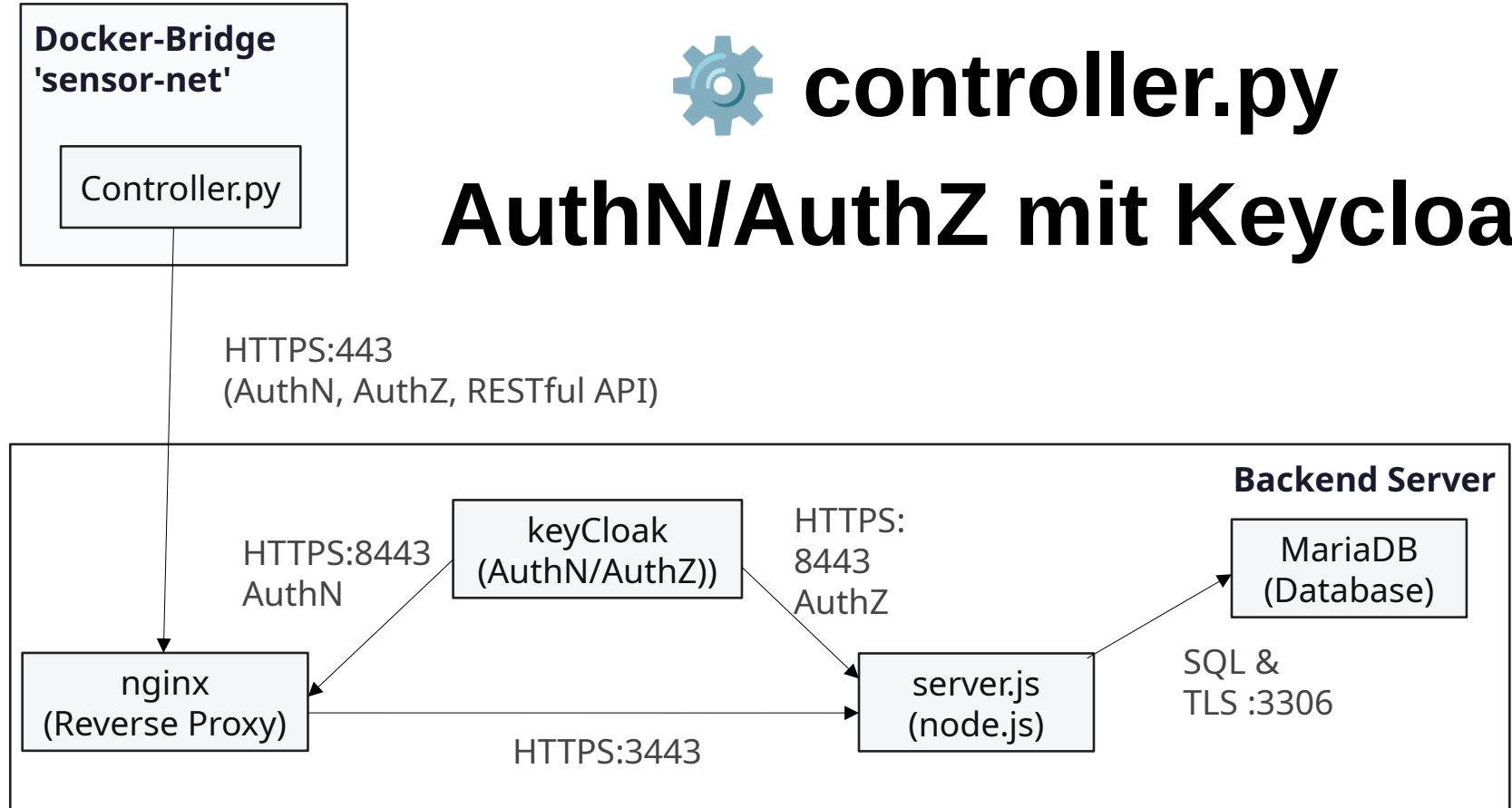
② Vergleich: welcher Endpoint nutzt welche Auth-Middleware

```
POST /api/internal/sensordata
POST /api/internal/trainingdata
GET /api/internal/sensordata/latest
GET /api/sensordata
```



controller.py

AuthN/AuthZ mit Keycloak



sensor-net > http_client.py > Inline-Header → _build_headers() · verify=False → CA_CERT_PATH

http_client.py (before)

① Imports: tensorflow entfernt, keycloak_auth + CA_CERT_PATH neu
3 import requests
4- from tensorflow...tag_constants import TRAINING
6- from config import BACKEND_URL, API_KEY, HTTP_TIMEOUT,
log, TRAININGURL

② Neue Funktion _build_headers() – zentraler Header mit API-Key + Bearer-Token

③ forward_to_backend() inline headers → _build_headers() · verify=False → CA_CERT_PATH

```
18-     headers = {"Content-Type": "application/json", "x-  
api-key": API_KEY}  
26-     headers=headers,  
28-     verify=False, # Until nginx Certificate can be  
verified  
58-     headers = {"Content-Type": "application/json", "x-  
api-key": API_KEY }  
66-     headers=headers,  
68-     verify=False,
```

after

① Imports: tensorflow entfernt, keycloak_auth + CA_CERT_PATH neu
3 import requests
5+ from config import BACKEND_URL, API_KEY, HTTP_TIMEOUT,
CA_CERT_PATH, log
6+ from keycloak_auth import auth_header
9+ def _build_headers() -> dict:
10+ """Erstellt die HTTP-Header mit API-Key und Bearer-
Token."""

② Neue Funktion _build_headers() – zentraler Header mit API-Key + Bearer-Token

```
11+     headers = {  
12+         "Content-Type": "application/json",  
13+         "x-api-key": API_KEY,  
15+         headers.update(auth_header())  
16+     }  
    return headers
```

③ forward_to_backend() · inline headers → _build_headers() · verify=False → CA_CERT_PATH

```
32+     headers=_build_headers(),  
34+     verify=CA_CERT_PATH,  
68+     headers=_build_headers(),  
70+     verify=CA_CERT_PATH,
```

keycloak_auth.py (before)

Komplett neue Datei – Keycloak Client-Credentials-Flow für den Controller

- ① Token-Cache – Module-level globals für Access-Token und Ablaufzeit
- ② request_token() – POST Client-Credentials → Keycloak, Access-Token cachen (~30s Puffer)

after

Komplett neue Datei – Keycloak Client-Credentials-Flow für den Controller

```
1+ """Keycloak Client-Credentials-Flow: Token holen, cachen, Rolle prüfen."""
3+ import base64, json, time
7+ import requests
9+ from config import KC_TOKEN_URL, KC_CLIENT_ID, KC_CLIENT_SECRET, KC_REQUIRED_ROLE, CA_CERT_PATH, ...
14+ _cached_token: str | None = None
15+ _token_expires_at: float = 0.0
② request_token() – POST Client-Credentials → Keycloak, Access-Token cachen (~30s Puffer)
26+ def request_token() -> str:
28+     data = {"grant_type": "client_credentials", "client_id": KC_CLIENT_ID, ...}
33+     resp = requests.post(KC_TOKEN_URL, data=data, timeout=HTTP_TIMEOUT, verify=CA_CERT_PATH)
43+     _cached_token = body["access_token"]
45+     _token_expires_at = time.time() + body.get("expires_in", 300) - 30 # 30s Puffer
```

Sensor-net > keycloak_auth.py > Client-Credentials-Flow, Token-Cache, Rollenprüfung [Teil 2/2]

keycloak_auth.py (before)

Komplett neue Datei – Keycloak Client-Credentials-Flow für den Controller

③ `get_token()` – Cache-Check · `verify_role()` – Rollenprüfung · `auth_header()` – Bearer-Header

after

③ `get_token()` – Cache-Check · `verify_role()` – Rollenprüfung · `auth_header()` – Bearer-Header

```
51+ def get_token() -> str: # cached oder frisch
53+     if _cached_token and time.time() < _token_expires_at: return
                                     _cached_token

58+ def verify_role() -> None: # RuntimeError wenn Rolle fehlt
65+     roles = payload.get("realm_access", {}).get("roles", [])
66+     if KC_REQUIRED_ROLE not in roles: raise RuntimeError(...)
76+ def auth_header() -> dict: # {"Authorization": "Bearer <token>"}
```

Sensor-net > controller.py > Keycloak-Integration

controller.py (before)

```
① Import: CA_CERT_PATH + verify_role / auth_header aus
keycloak_auth
21- MQTT_KEEPALIVE, API_KEY, BACKEND_URL, HTTP_TIMEOUT,
log,

② warmstart() - auth_header() zu bestehenden Headers
hinzufügen · verify=CA_CERT_PATH
35 headers = {"Content-Type": "application/json", "x-
api-key": API_KEY}
41- url, headers=headers, timeout=HTTP_TIMEOUT,
verify=False

③ main() - Keycloak AuthN/AuthZ: verify_role() vor warmstart()
75 if not os.path.isfile(os.getenv("MQTT_CA_FILE",
"")): ...
```

after

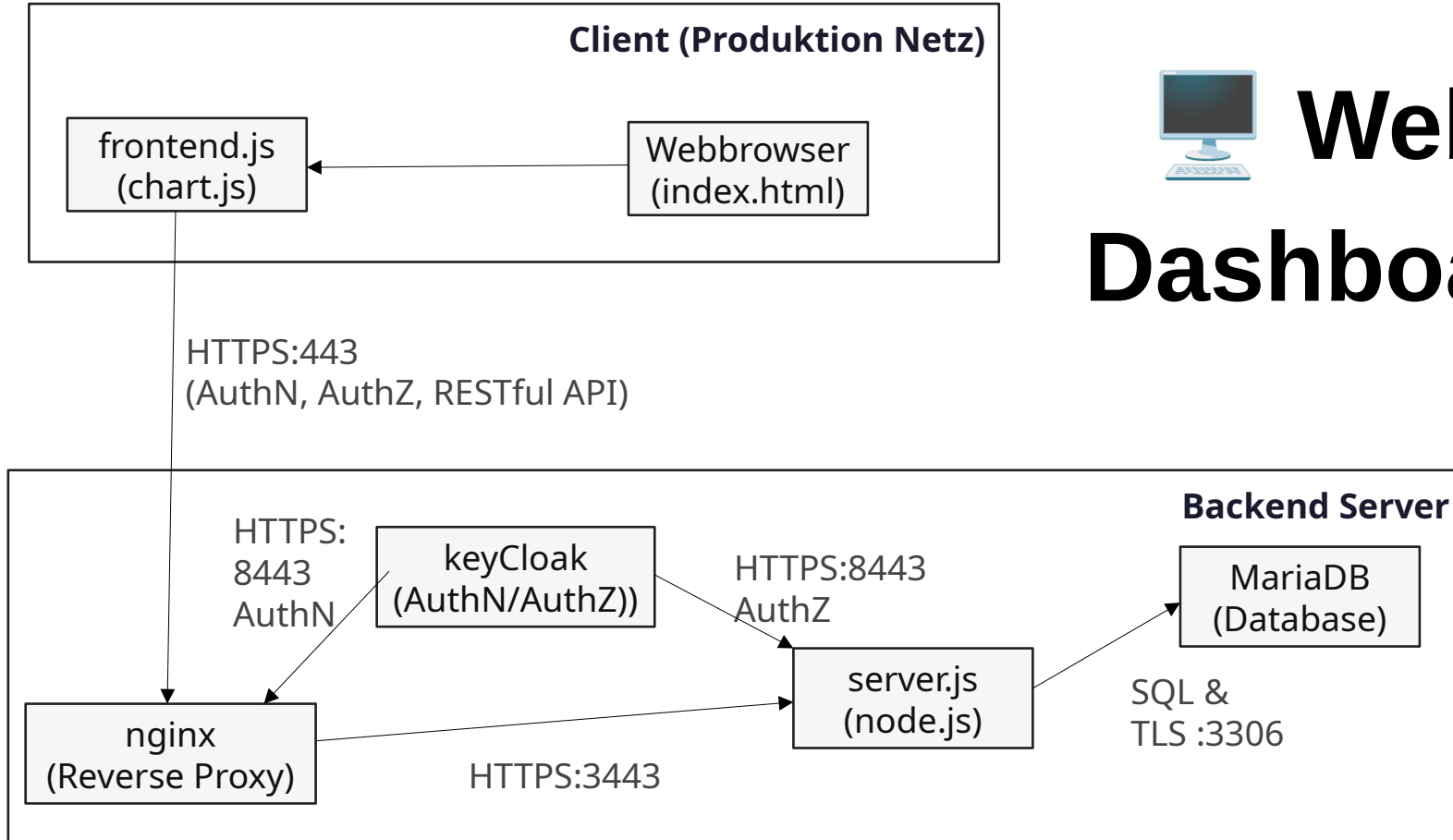
```
① Import: CA_CERT_PATH + verify_role / auth_header aus
keycloak_auth
21+ MQTT_KEEPALIVE, API_KEY, BACKEND_URL, HTTP_TIMEOUT,
22+ CA_CERT_PATH, log,
24+ from keycloak_auth import verify_role, auth_header

② warmstart() - auth_header() zu bestehenden Headers
hinzufügen · verify=CA_CERT_PATH
38 headers = {"Content-Type": "application/json", "x-
api-key": API_KEY}
44+ url, headers=headers, timeout=HTTP_TIMEOUT,
verify=CA_CERT_PATH

③ main() - Keycloak AuthN/AuthZ: verify_role() vor warmstart()
78 if not os.path.isfile(os.getenv("MQTT_CA_FILE",
"")): ...
81+ # Keycloak AuthN/AuthZ: Token holen und Rolle prüfen
82+ try:
83+     verify_role()
84+ except Exception as exc:
85+     log.error("Keycloak-Authentifizierung
fehlgeschlagen: %s", exc)
86+     sys.exit(1)
```



Web-Dashboard



```
1 <!DOCTYPE html>
2 <html lang="de">
3 <head> ...
9 </head>
10
11 <body>
12 <header class="topbar"> ...
26 </header>
27
28 <main class="container"> NussWal, 13 hours ago • Updated Web-Dashboard fix ...
29 <section class="card controls">
30 <h2>Zeitraum wählen</h2>
31
32 <div class="input-row horizontal">
33 <label>
34 Von:
35 <input type="datetime-local" id="from">
36 </label>
37
38 <label>
39 Bis:
40 <input type="datetime-local" id="to">
41 </label>
42
43 <button id="loadDataBtn">Daten laden</button>
44 <button id="logoutBtn" class="logout-btn">Logout</button>
45 </div>
46 </section>
47
48 <section class="card chart-card">
49 <h2>Sensordaten</h2>
50 <div id="chartStatus"></div>
51 <canvas id="sensorChart"></canvas>
52 </section>
53 </main>
54
55 <script src="https://cdn.jsdelivr.net/npm/keycloak-js@23.0.7/dist/keycloak.min.js"></script>
56 <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
57
58 <script type="module" src="frontend.js"></script>
59 <script type="module" src="chart.js"></script>
60 </body>
61 </html>
```


Zeitraum wählen

Von: 

Bis: 

Daten laden

Logout

Sensordaten

webapp > frontend.js [Teil 1/2]

```
1  const Keycloak = window.Keycloak;
2
3  const keycloak = new Keycloak({
4    url: "https://192.168.4.18/auth",
5    realm: "iot",
6    clientId: "dashboard-client"
7  });
8
9  async function initAuth() {
10    try {
11      const authenticated = await keycloak.init({
12        onLoad: "login-required",
13        checkLoginIframe: false
14      });
15
16      > if (!authenticated) { ...
17
18      }
19
20      document.getElementById("userInfo").textContent =
21        "Eingeloggt als: " + keycloak.tokenParsed.preferred_username;
22
23      setInterval(() => {
24        keycloak.updateToken(30).catch(() => {
25          console.error("Token refresh failed");
26          keycloak.login();
27        });
28      }, 30000);
29
30    } catch (err) {
31      console.error("Keycloak init error:", err);
32    }
33  }
34
35
```

```

36 async function apiGet(path) {
37   const url = `https://www.lab.local/api${path}`;
38
39   const response = await fetch(url, {
40     method: "GET",
41     headers: {
42       "Authorization": "Bearer " + keycloak.token
43     }
44   });
45
46 > if (!response.ok) { ...
47   }
48
49
50   return response.json();
51 }
52
53 export async function getSensorRange(from, to) {
54   const query = `?from=${encodeURIComponent(from)}&to=${encodeURIComponent(to)}`;
55   return apiGet(`/sensordata/range${query}`);
56 }
57
58 document.getElementById("loadDataBtn").addEventListener("click", () => {
59   const from = document.getElementById("from").value;
60   const to = document.getElementById("to").value;
61
62 > if (!from) { ...
63   }
64
65
66   window.dispatchEvent(new CustomEvent("loadRange", { detail: { from, to } }));
67 });
68
69
70 // --- Dark Mode Toggle (ohne Speicherung) ---
71 const themeToggle = document.getElementById("themeToggle");
72 > themeToggle.addEventListener("change", () => { ...
73   });
74
75
76 // --- Logout Button ---
77 document.getElementById("logoutBtn").addEventListener("click", () => {
78   keycloak.logout();
79 });
80
81
82 initAuth();

```

```
1  import { getSensorRange } from "../frontend.js";  
2  
3  const Chart = window.Chart;  
4  
5  let chart = null;  
6  let liveUpdateInterval = null;  
7  
8  // Statusmeldung anzeigen  
9  function setStatus(message, type = "info") {  
10     const el = document.getElementById("chartStatus");  
11     if (!el) return;  
12  
13     el.textContent = message;  
14     el.className = "";  
15     el.classList.add(`status-${type}`);  
16 }  
17
```

```
18 async function renderRange(from, to) {
19   try {
20     const data = await getSensorRange(from, to);
21
22     if (!data.labels || data.labels.length === 0) {
23       if (chart) chart.destroy();
24       setStatus("Keine Daten im gewählten Zeitraum", "error");
25       return;
26     }
27
28     const ctx = document.getElementById("sensorChart").getContext("2d");
29
30     if (chart) chart.destroy();
31
32     chart = new Chart(ctx, {
33       type: "line",
34       data: {
35         labels: data.labels,
36         datasets: [ ...
55       ]
56     },
57     options: { ...
67   }
68 });
69
70   if (to) {
71     setStatus(`Statischer Zeitraum geladen, ${data.labels.length} Rohdatensätze`, "success");
72   }
73
74 } catch (err) {
75   console.error("Chart render error:", err);
76   setStatus("Verbindungsproblem, Daten konnten nicht geladen werden", "error");
77 }
78 }
```

```
80 window.addEventListener("loadRange", (e) => {
81     const { from, to } = e.detail;
82
83     if (liveUpdateInterval) {
84         clearInterval(liveUpdateInterval);
85         liveUpdateInterval = null;
86     }
87
88     if (!to) {
89         setStatus("Live-Modus aktiv (minütliche Aktualisierung)", "live");
90
91         renderRange(from, null);
92
93         liveUpdateInterval = setInterval(() => {
94             renderRange(from, null);
95             setStatus(`Live-Modus aktualisiert: ${new Date().toLocaleTimeString()}`, "live");
96         }, 60000);
97     } else {
98         renderRange(from, to);
99     }
100 }
101 });
```

NussWal, 19 hours ago • Web-Dashboard possibly final style adjustments ..

IOT

We are sorry...

Invalid parameter: redirect_uri



Fazit

Verschlüsselung zu MQTT: funktioniert ✓

Reverse Proxy (nginx): HTTPS → HTTP — Problem mit Zertifikaten

Routing und Keycloak Authentifikation → Problem im Browser